

Lab 1.2: Design Principles Review Report

Group: Group 25 | Course: Software Design and Construction (ISD.20252)

1. Single Responsibility Principle (SRP)

Modules	PIC	Description	Improvement Direction
<code>products.component.ts</code> (Frontend)	Nguyễn Hoàng Trung Kiên	The component handles too many logic flows: Fetching data, Pagination, Form validation, Modal visibility, API management, and stock adjustment business logic.	Extract form validation into a separate Validator Service. Move state management logic to facade services.
<code>payment.component.ts</code> (Frontend)	Ngô Đức Huy, Đặng Hoàng Sơn	Contains UI routing logic and loading states, while directly parsing callback URLs and calling the PayPal capture API.	Separate the URL callback parsing and payment API calls into a dedicated PaymentProcessorService.
<code>orders.component.ts</code> (Frontend)	Bùi Doãn Khang	Manages tab switching, confirm/alert popups, and the order approval/rejection flow directly within the UI Component file.	Extract Modal showing functions into a parent Component or a shared Service. Separate order processing business logic into an Application Service.
<code>users.component.ts</code> (Frontend)	Lê Kế Đức	Calls HttpClient directly without going through a UserService. Handles filtering logic locally on the client.	Build a UserService to wrap HttpClient. The filtering logic should be moved to the Server or extracted into a utility function.

2. Open/Closed Principle (OCP)

Modules	PIC	Description	Improvement Direction
<code>products.component.ts</code> (Frontend)	Nguyễn Hoàng Trung Kiên	<code>validateForm()</code> and <code>saveProduct()</code> logic use rigid if-else structures to check <code>mediaType</code> (BOOK, CD, DVD, NEWSPAPER). Adding a new type will break existing code.	1. Define a <code>ProductForm interface</code>. 2. Implement concrete <code>classes</code>: <code>BookForm</code>, <code>CdForm</code>,... that implement <code>ProductForm</code> to separate <code>validate/payload</code> logic for each product type. 3. Create a <code>ProductFormResolver class</code> to register and resolve the corresponding form.

Modules	PIC	Description	Improvement Direction
<code>payment.component.ts</code> (Frontend)	Ngô Đức Huy, Đặng Hoàng Sơn	Hardcoded condition if (<code>paymentMethod === 'QR'</code>) { ... } else { ... } in the <code>confirmPayment()</code> function.	1. Define an <code>IPayment interface</code>. 2. Implement concrete <i>classes</i>: <code>PaypalPayment</code>, <code>VietQrPayment</code>,... that implement <code>IPayment</code>. 3. Create a <code>PaymentResolver class</code> containing a <code>Map<string, IPayment></code> configuration to flexibly extend other payment methods.
<code>users.component.ts</code> (Frontend)	Lê Kế Đức	The list of roles availableRoles = ['ADMIN', 'PRODUCT_MANAGER', 'STAFF'] is hardcoded directly into the UI source code.	Dynamically fetch the available roles list via an API provided by the backend.
<code>product.service.ts</code> (Backend)	Nguyễn Hoàng Trung Kiên	The <code>saveProductDetail</code> and <code>findDetail</code> functions use <code>switch(mediaType)</code> for DB operations. It is very difficult to extend to new Media types.	1. Define a <code>MediaDetailHandler interface</code>. 2. Implement concrete <i>classes</i>: <code>BookDetailHandler</code>, ... that implement <code>MediaDetailHandler</code> to handle their own separate storage business logic. 3. Create a <code>MediaDetailResolver class</code> to return the corresponding handler.
<code>shipping-calculator.service.ts</code> (Backend)	Bùi Doãn Khang	The formula for <code>calculateInnerCityFee</code> vs <code>calculateOtherProvinceFee</code> is fixed.	1. Define a <code>ShippingFee interface</code>. 2. Implement concrete <i>classes</i>: <code>InnerCityShipping</code>, <code>OtherProvinceShipping</code>,... that implement <code>ShippingFee</code> to separate fee calculation formulas. 3. Create a <code>ShippingFeeResolver class</code> to resolve the corresponding fee calculation class.
<code>order.service.ts</code> (Backend)	Ngô Đức Huy, Đặng Hoàng Sơn	Manually checking the payment gateway (PAYPAL, VIETQR) with rigid if-else structures to refund in <code>rejectOrder()</code> and <code>cancelOrder()</code> .	Define a <code>RefundProcessor interface</code> and implement concrete <i>classes</i> for each payment gateway.
<code>product-validator.factory.ts</code> (Backend)	Nguyễn Hoàng Trung Kiên	The map containing validators (validators map) is statically hardcoded in the source code with 4 fixed product types.	Inject a dynamic list of validators through NestJS's custom Injection Token.

3. Dependency Inversion Principle (DIP)

Modules	PIC	Description	Improvement Direction
<code>users.component.ts</code> (Frontend)	Lê Kế Đức	The frontend component depends directly on <code>HttpClient</code> for data manipulation instead of through an Abstraction layer.	Create data services belonging to abstractions (e.g., <code>IUserService</code>) to manage network requests independently.
<code>order.service.ts</code> (Backend)	Đặng Hoàng Sơn	Depends directly on the concrete class <code>PaypalService</code> (causing a circular dependency error and requiring <code>forwardRef</code>).	Inject through an abstraction <i>interface</i> instead of a concrete class.

4. Interface Segregation Principle (ISP)

Modules	PIC	Description	Improvement Direction
<code>products.component.ts</code> (Frontend)	Nguyễn Hoàng Trung Kiên	The <code>productForm</code> object simultaneously contains properties of book, cd, dvd, newspaper (junk fields). It forces the component to carry unnecessary fields for a specific media type.	Define separate Form Interfaces for each distinct Product type (<code>IBookForm</code> , <code>ICDForm</code>).